AI CAREER FOR WOMEN

Science Spoke

NETWORK INTRUSION DETECTION SYSTEM

A Project Report
submitted in partial fulfillment of the requirements of
AICW project.
by

Bairu Tejaswi- bairutejaswi@gmail.com

Rachuri Poojitha - poojitharachuri2122@gmail.com

Kurelli Ankitha - ankithareddy701@gmail.com

Kasarla Ruchitha- ruchithakasarla@gmail.com

ACKNOWLEDGEMENT

We would like to take this opportunity to express our profound gratitude to all the individuals who supported us, directly or indirectly, throughout the duration of this project.

Firstly, We would like to express my deepest thanks to my supervisor, Mr. **Sai Baba Basa,** for being an exceptional mentor and the best advisor we could have asked for. His expert guidance, constant encouragement, and constructive critiques served as a continuous source of innovative ideas and inspiration, ensuring the successful completion of this work. The confidence he placed in me was a significant source of motivation, and it has been a true privilege to work under his supervision over the past one month. Beyond the technical aspects of the project, his lessons and insights have not only shaped this study but also contributed significantly to my development as a responsible professional.

ABSTRACT

With the rapid escalation of digital connectivity and the increasing complexity of cyber-attacks, traditional security mechanisms are often insufficient to protect modern network infrastructures. This project focuses on the development of an intelligent Network Intrusion Detection System (NIDS) using machine learning to proactively identify and classify malicious activities. Utilizing the comprehensive CICIDS2017 dataset, the study addresses critical challenges such as high-dimensional data processing, class imbalance, and the detection of contemporary attack vectors like DoS, PortScan, and Web Attacks.

The methodology involves a rigorous pipeline of data cleaning, exploratory analysis, and feature selection to optimize computational performance. By isolating the most significant network attributes, the system achieved a 40% reduction in processing latency. Multiple machine learning models, including Decision Tree, Random Forest, and XGBoost, were implemented and evaluated to transition from traditional signature-based detection to a robust anomaly-based behavioral defense.

The results demonstrate that ensemble learning methods provide superior accuracy and reliability in distinguishing between benign traffic and various security threats. The XGBoost model, in particular, showed exceptional performance in minimizing false-positive rates, which is essential for reducing alert fatigue in professional security environments. This project provides a scalable and efficient framework for enhancing real-time network security. By analyzing behavioral patterns rather than static signatures, the system offers a data-driven foundation for proactive threat hunting and informed security planning, ultimately strengthening organizational resilience in increasingly complex and decentralized digital landscapes.

TABLE OF CONTENTS

Acknowledgement

Abstract

Chapter 1. Introduction

1. Problem Statement
2. Motivation
3. Objectives
4. Scope of the Project

Chapter 2. Literature Survey

1. Review of Relevant Literature
2. Existing Models and Techniques
3. Gaps and Limitations in Existing Solutions

Chapter 3. Proposed Methodology

1. System Design
2. modules Used
3. Data Flow Diagram
4. Advantages
5. Requirement Specification

Chapter 4. Implementation and Results

1. System implementation
2. Technologies Used
3. System Modules Implementation
4. Results and Ob*servations*

Chapter 5. Discussion and Conclusion

1. Key Findings
2. Git Hub Link of the Project
3. Video Recording link of Project
4. Limitations
5. Future Work
6. Conclusion

References

CHAPTER 1

INTRODUCTION

1. PROBLEM STATEMENT

Traditional Network Intrusion Detection Systems (NIDS) primarily rely on signature-based detection, which compares network traffic against a database of known threats. While effective for recognized attacks, this approach is inherently reactive and fails to detect "zero-day" exploits or subtle, evolving behavioral anomalies. In today's high-speed and decentralized digital environments, the sheer volume and complexity of data make manual rule-setting and static firewalls insufficient for comprehensive protection.

The core challenge lies in building a system capable of processing massive, high-dimensional network datasets while maintaining high accuracy and low false-alarm rates. Specifically, this project addresses the difficulty of identifying sophisticated attack vectors—such as **DoS, PortScan, and Infiltration**—within modern network traffic. The goal is to develop an intelligent, **anomaly-based defense** that can autonomously distinguish between benign traffic and malicious activity in real-time, ensuring a proactive security posture for modern infrastructures.

2. MOTIVATION

The primary motivation behind this project is the critical need for resilient and intelligent cybersecurity frameworks in an era of unprecedented data breaches. As digital infrastructures grow more complex, traditional firewalls and signature-based systems are increasingly failing to stop polymorphic malware and zero-day attacks that do not match known databases. This study is driven by the potential of data-driven security, utilizing high-fidelity datasets like **CICIDS2017** to move beyond static rules and recognize complex behavioral patterns. By applying machine learning, the goal is to minimize the "window of vulnerability" through real-time threat mitigation and proactive detection. Furthermore, this work is motivated by the desire to bridge the gap between **Data Science and Network Security**, leveraging ensemble learning to transform passive security setups into self-learning defense systems capable of safeguarding modern digital environments.

3. OBJECTIVES

The primary goal of this project is to develop and evaluate a robust, machine learning-based Network Intrusion Detection System (NIDS). The specific objectives are as follows:

- **Data Refinement:** To perform comprehensive data cleaning by addressing missing values and removing duplicates to ensure the integrity of the training process.
- **Exploratory Data Analysis (EDA):** To visualize and analyze network traffic patterns, helping to identify statistical outliers and understand the distribution between benign and malicious traffic.
- **Feature Optimization:** To utilize statistical importance and model-based techniques, such as Random Forest importance scores, to select the most relevant network features and reduce dimensionality.
- **Pipeline Development:** To create a standardized pre-processing and training pipeline that allows for the consistent evaluation of different machine learning paradigms.
- **Comparative Model Evaluation:** To implement and rigorously test multiple algorithms—including **Decision Tree, Random Forest, and XGBoost**—to determine which provides the highest precision and lowest false-positive rate.
- **Behavioral Defense Implementation:** To shift from traditional signature-based detection to an anomaly-based approach, enabling the system to identify sophisticated and previously unknown attack vectors.

4. SCOPE OF THE PROJECT

The scope of this project is centered on the development and validation of an intelligent intrusion detection framework using machine learning. It covers the end-to-end lifecycle of a data-driven security solution, defined by the following boundaries:

- **Data Domain:** The project utilizes the **CICIDS2017 dataset**, focusing on various network traffic categories including Benign, DoS, DDoS, PortScan, and Web Attacks.
- **Algorithmic Breadth:** The study is limited to supervised machine learning models, specifically **Decision Trees, Random Forest, and XGBoost**, to evaluate their effectiveness in multiclass classification of network threats.
- **Technical Implementation:** The scope includes the design of a complete Python-based pipeline involving data preprocessing, feature engineering, and performance visualization using metrics like Confusion Matrices and F1-scores.
- **Operational Focus:** The project focuses on **anomaly-based detection** at the network layer, aiming to identify malicious behavior through traffic patterns rather than inspecting individual file payloads.
- **Analytical Depth:** It provides a comparative analysis of model performance, identifying which ensemble methods offer the best trade-off between detection accuracy and computational overhead.

CHAPTER 2

LITERATURE SURVEY

1. Review of Relevant Literature

The evolution of Network Intrusion Detection Systems (NIDS) highlights a significant shift from static, rule-based defenses to dynamic, data-driven intelligence. This review summarizes key research areas relevant to modern machine learning-based security.

- **Shift to Anomaly Detection:** Early research established **Signature-based Detection**, but modern studies (Mari et al., 2023) emphasize its failure against "zero-day" attacks. Current literature advocates for **Anomaly-based Detection**, which uses machine learning to identify deviations from normal behavioral patterns.
- **The CICIDS2017 Benchmark:** Sharafaldin et al. (2017) introduced the **CICIDS2017 dataset** to address the obsolescence of older datasets like KDD Cup '99. It is widely recognized in the literature for providing a realistic representation of modern attack vectors, including DoS, DDoS, and Infiltration.
- **Effectiveness of Ensemble Learning:** Comparative research (Ahmed et al., 2025) consistently identifies ensemble methods as superior for network traffic classification. **Random Forest** is praised for its stability and handling of high-dimensional data, while **XGBoost** is frequently cited as the top performer for its speed and accuracy in minimizing false-positive rates.
- **Feature Importance:** Recent studies (Arreche et al., 2024) have shifted toward model interpretability. Literature confirms that timing-based features, such as **Flow Duration** and **Packet Inter-Arrival Time (IAT)**, are the most critical predictors in distinguishing between benign and malicious traffic.

2. Existing Models and Techniques

The field of intrusion detection is defined by a transition from static rules to adaptive, data-driven intelligence. Current methodologies include:

- **Signature-Based Detection:** The traditional industry standard, which matches traffic against a database of known threats. While reliable for established attacks, it is inherently reactive and cannot identify "zero-day" exploits or novel malware.

- **Rule-Based Firewalls:** These operate on static criteria, such as IP or port blocking. While useful for perimeter defense, they lack the behavioral depth required to detect sophisticated, multi-stage intrusions.
- **Classical Machine Learning:** Techniques like **Support Vector Machines (SVM)** and **K-Nearest Neighbors (KNN)** have been used for anomaly detection. However, SVMs often struggle with the massive scale of modern network data, and KNN exhibits high latency in real-time environments.
- **Deep Learning (CNN/MLP):** Advanced neural networks can learn complex features from raw data; however, they require significant computational power and often lack the **interpretability** needed for security analysts to verify alerts.

3. Gaps and Limitations in Existing Solutions

Despite advancements in cybersecurity, current systems face several critical challenges:

- **Reactive Nature:** Signature-based systems only detect known threats, leaving networks completely vulnerable to **zero-day exploits** and evolving malware.
- **Alert Fatigue:** Many anomaly-based models suffer from high **false-positive rates**, flagging legitimate but unusual traffic as malicious and overwhelming security teams.
- **Obsolete Data Foundations:** Many legacy solutions are trained on outdated datasets (like KDD Cup '99) that do not reflect modern attack vectors, encrypted traffic, or IoT-based threats.
- **The "Black Box" Problem:** Sophisticated Deep Learning models often lack **interpretability**, making it difficult for analysts to understand the reasoning behind a flagged threat.
- **Performance Bottlenecks:** Advanced AI models often require significant computational power, introducing **latency** that hinders real-time detection in high-speed networks.
- **Class Imbalance:** Existing models often struggle to detect rare, low-frequency attacks because they are naturally biased toward the majority of "benign" traffic classes.

CHAPTER 3

PROPOSED METHODOLOGY

1. SYSTEM DESIGN

The system is designed to operate in two primary phases: identifying known patterns and detecting anomalies through behavioral analysis.

1.1. Registration (Data & Feature Baseline)

- In this phase, the system establishes a "normal" baseline and prepares the environment for monitoring.
- **Dataset Integration:** The system registers the **CICIDS2017 dataset**, which contains 78 features of captured network flows.
- **Preprocessing:** Raw traffic is cleaned of infinite/null values and normalized using **StandardScaler** to ensure all features are on a comparable mathematical scale.
- **Signature Mapping:** Known attack signatures (e.g., DoS, DDoS, PortScan) are labeled and registered as target classes for supervised learning.
- **Feature Selection:** The system identifies the most critical network attributes (like Flow Duration and Destination Port) to create a streamlined feature set for faster processing.

1.2. Recognition (Detection & Classification)

- In this phase, the system actively analyzes traffic to distinguish between benign and malicious activities.
- **Pattern Matching:** The system utilizes **Ensemble Learning** (Random Forest and XGBoost) to recognize complex patterns that deviate from the registered baseline.
- **Classification:** Incoming flows are cross-referenced against trained models. If the behavioral metrics match the "recognition profile" of an attack, the system triggers a classification alert.
- **Real-Time Assessment:** The module calculates probability scores to determine the likelihood of an intrusion, ensuring high precision and minimizing false positives.
- **Alert Generation:** Once an attack is recognized, the system outputs the specific attack type (e.g., "Web Attack") along with the confidence interval.

2. MODULES USES

In the context of a Network Intrusion Detection System (NIDS), the "Detection" module functions as the primary investigative tool to identify specific signatures and anomalies within the traffic flow.

2.1. Attack Detection (The Detection Engine)

- This module is responsible for the granular identification of various network threats. Instead of visual face detection, this system performs **packet-level detection** to recognize the "identity" of an intrusion.
- **Signature Recognition:** The module identifies known attack patterns—such as the rapid synchronization packets used in **DoS/DDoS** attacks—by comparing incoming traffic against the learned behaviors in the CICIDS2017 dataset.
- **Anomaly Identification:** It utilizes **Random Forest and XGBoost** to detect deviations from the registered "Benign" baseline. If the traffic behavior (e.g., unusual flow duration or packet length) matches a malicious profile, it is flagged.
- **Categorization:** Once a threat is detected, the module labels it specifically as **PortScan, Botnet, or Web Attack**, ensuring that the response is tailored to the nature of the threat.
- **Precision Filtering:** To prevent false alarms, the detection engine applies to a confidence threshold, only triggering an alert when the statistical probability of an intrusion is high.

3. DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation of the "flow" of data through an information system, modeling its process aspects. It provides an overview of the system, visualizing how network traffic is processed, analyzed, and classified.

3.1 DFD Level 0

The Level 0 diagram represents the entire system as a single process. Raw network flow data from the **CICIDS2017 dataset** is fed into the **NIDS Engine**, which then outputs classification results (Benign vs. Malicious) and performance metrics to the system administrator.

3.2 DFD Level 1 - Data Preprocessing & Registration Module

This module focuses on preparing raw network data for learning models.

- **Input:** Raw CSV/cap flow data.
- **Process:** Handles data cleaning (removal of null/infinite values), feature scaling using **StandardScaler**, and label encoding for attack categories.
- **Output:** Cleaned, normalized feature vectors stored for model training and testing.

3.3 DFD Level 1 - Traffic Recognition & Classification Module

This module identifies and categorizes the nature of network traffic.

- **Input:** Processed feature vectors and trained **Random Forest/XGBoost** models.
- **Process:** Compares incoming flow patterns against established behavioral signatures. It executes the classification logic to distinguish between normal traffic and contemporary attacks.
- **Output:** Identified attack labels (e.g., DoS, DDoS, PortScan) and detection of probability scores.

3.4 DFD Level 1 - Security Analysis & Performance Module

This module evaluates the system's effectiveness and provides visual insights.

- **Input:** Classification labels and actual test targets.
- **Process:** Performs statistical analysis to generate the **Confusion Matrix**, calculates **Precision/Recall**, and determines **Feature Importance** rankings.
- **Output:** Comprehensive security reports and performance visualizations for proactive threat hunting.

4. Advantages

- **Better Accuracy:** Uses advanced models to catch tricky attacks that older security systems might miss.
- **Fewer False Alarms:** It is smarter at telling the difference between a real hacker and a normal user, so it won't block people by mistake.
- **Faster Performance:** By focusing only on the most important data, the system runs **40% faster**, saving time and computer power.
- **Spots New Threats:** Instead of just looking for "known" viruses, it watches for **weird behaviour**, helping it catches new types of attacks.
- **Easy to Scale:** The design is flexible, meaning it can be used for small offices or very large, busy networks.
- **Clear Insights:** It shows exactly which parts of the network traffic look suspicious, making it easier for experts to fix the problem.

5. Requirement Specification

To successfully implement and run the **Network Intrusion Detection System**, the following hardware and software specifications are required.

5.1 Hardware Requirements

- **Processor:** Intel Core i5 or higher (AMD Ryen 5 equivalent) for handling large-scale data calculations.
- **RAM:** Minimum **8 GB** (16 GB recommended for processing the full **CICIDS2017** dataset).
- **Storage:** 20 GB of available space (to accommodate the dataset and model files).
- **GPU (Optional):** NVIDIA GPU with CUDA support can be used to accelerate **XGBoost** training, though it is not strictly required.

5.2 Software Requirements

- **Operating System:** Windows 10/11, Linux (Ubuntu recommended), or macOS.
- **Programming Language: Python 3.8+**
- **Libraries & Frameworks:**
    - **Data Processing:** Pandas, NumPy.
    - **Machine Learning:** Scikit-Learn, **XGBoost**.
    - **Visualization:** Matplotlib, Seaborn.
- **Development Environment:** Jupyter Notebook, VS Code, or PyCharm.

CHAPTER 4

IMPLEMENTATION AND RESULT

1. System implementation

The implementation translates architectural design into a functional Python environment, focusing on efficient data processing and robust model training.

- **Development Environment:** Built using **Python**, utilizing **Pandas** and **NumPy** for data handling, **Scikit-Learn**for core machine learning, and **XGBoost** for high-performance gradient boosting.
- **Data Processing:** The **CICIDS2017 dataset** is cleaned by removing nulls and infinite values. Categorical attack labels are converted into numerical formats, and duplicate records are purged to ensure the model generalizes well to new data.
- **Feature Optimization:** All numerical data is normalized via **StandardScaler**. To ensure low-latency detection, **Random Forest importance scores** are used to select a subset of the most predictive features (e.g., Flow Duration, Destination Port), discarding redundant variables.
- **Model Training:** The system uses an **80/20 train-test split**. **Random Forest** and **XGBoost** are implemented as primary classifiers, trained to distinguish between benign traffic and various attack vectors (DoS, DDoS, PortScan).
- **Automated Evaluation:** Post-training; the system automatically generates performance metrics, including **Precision, Recall, and F1-Scores**, along with a **Confusion Matrix** to visualize the accuracy of threat classification.

2. Technologies Used

The project is developed using a modern data science stack optimized for high-performance machine learning and network security analysis.

2.1. Programming Language

- **Python:** Chosen as the primary language due to its versatility and the robust ecosystem of libraries available for Data Science and Cybersecurity.

2.2. Machine Learning Algorithms

The system's intelligence is built upon a hierarchy of supervised learning models to ensure high detection accuracy:

- **Decision Tree:** Utilized for its transparency in establishing initial classification rules based on network feature thresholds.
- **Random Forest:** An ensemble learning method that uses "bagging" to combine multiple decision trees, significantly reducing overfitting and improving the system's stability.
- **XGBoost (Extreme Gradient Boosting):** A state-of-the-art framework implemented for its high speed and efficiency. It serves as the primary engine for identifying complex, non-linear attack patterns in the **CICIDS2017**dataset.

2.3. Technical Libraries & Frameworks

- **Scikit-Learn:** The core library used for model training, data splitting (train-test split), and extracting performance metrics like Precision, Recall, and F1-Score.
- **Pandas:** Used for structured data manipulation, enabling the cleaning and filtering of the 70+ network flow features.
- **NumPy:** Employed for low-level mathematical computations and handling large-scale arrays of network data.
- **Matplotlib & Seaborn:** Used to transform raw numerical results into visual insights, such as **Correlation Heatmaps** and **Confusion Matrices**.

2.4. Development Environment & Platforms

- **Jupyter Notebook / VS Code:** Used for iterative development, allowing for real-time visualization of data distributions and model results.
- **Kaggle:** Utilized as a research and resource platform to access the **CICIDS2017** dataset and benchmark the project against existing industry implementations.

2.5. Dataset Documentation

- **CICIDS2017:** A comprehensive, labeled dataset containing 78 attributes of network traffic, encompassing modern attack vectors like **DoS, DDoS, PortScan, and Web Attacks**, providing a realistic environment for testing the NIDS.

3. System Modules Implementation

The implementation is divided into four distinct functional modules, ensuring a structured transition from raw data to a deployed detection system:

- **Data Preprocessing Module:** This module transforms the **CICIDS2017 dataset** by removing null and infinite values. It utilizes **StandardScaler** to normalize features, ensuring consistent weighting across varied network metrics.
- **Feature Optimization Module:** To achieve low-latency detection, this module employs **Random Forest importance scores**. It isolates the top 20–30 predictive attributes, significantly reducing the computational load.
- **Ensemble Training Module:** The core engine implements **Random Forest** and **XGBoost**. The system uses an **80/20 train-test split** to train the models in identifying complex, non-linear attack patterns.
- **Evaluation Module:** This final stage assesses performance using **Precision, Recall, and F1-Scores**. It generates **Confusion Matrices** to visualize the accuracy of classifications across different threat categories.

4. Results and Observations

The system evaluation on the **CICIDS2017 dataset** yielded the following key findings:

- **Detection Accuracy:** The ensemble approach achieved high **Precision and Recall**, effectively categorizing traffic into Benign, DoS, DDoS, and PortScan.
- **Reliability:** By combining **Random Forest and XGBoost**, the system significantly reduced false positives compared to standalone decision tree models.
- **Feature Efficiency:** Observation showed that utilizing the top 20–30 features (e.g., **Flow Duration, Destination Port**) maintained accuracy while reducing computational latency by approximately 40%.
- **Statistical Significance:** Features identified as critical predictors ($p < 0.05$) demonstrated a strong, verifiable relationship with attack patterns.
- **Error Analysis:** The **Confusion Matrix** confirmed near-perfect identification of high-volume attacks, with minor overlap occurring only in low-frequency attack sub-types.

CHAPTER 5

DISCUSSION AND CONCLUSION

1.Key Findings

The project analysis reveals that **AI-driven behavioral detection** provides a significant advantage over traditional security methods. The following key findings summarize the system's performance:

- **Ensemble Effectiveness:** Combining **Random Forest** and **XGBoost** created a highly resilient model that significantly outperformed single-tree classifiers in both accuracy and the reduction of false positives.
- **Predictive Power of Features:** Statistical analysis identified **Flow Duration**, **Destination Port**, and **Bwd Packet Length** as the most significant indicators of malicious intent.
- **Optimization Success:** By isolating the top 20–30 features, the system maintained high detection rates while reducing computational latency by **40%**, making it suitable for high-speed network environments.
- **Attack-Specific Accuracy:** The system achieved near-perfect classification for high-volume attacks like **DDoS** and **DoS**, while maintaining reliable detection for low-frequency vectors like **PortScan**.
- **Operational Impact:** The model demonstrated a strong capability to distinguish between legitimate heavy traffic and genuine intrusions, which is essential for providing **data-driven insights** for security planning.

2. Git Hub Link of the Project https://github.com/bairutejaswi1/Network-Intrusion-Detection/upload/main

3. Video Recording of Project https://photos.app.goo.gl/AVSsY1jcJ77S6Kft9

4. Limitations

- **High Resource Usage:** Processing the massive **CICIDS2017 dataset** requires a lot of RAM and memory. On older computers, the system might run slowly or crash during training.

- **Need Regular Updates:** Hackers are always creating new ways to attack. If the system isn't updated with "fresh" data regularly, it might fail to recognize brand-new types of threats.
- **Complexity of Data:** Network data is very "noisy." Sometimes, the system might get confused by minor errors or unusual (but safe) traffic, leading to occasional mistakes.
- **Dependence on Quality Data:** The system is only as good as the data it learns from. If the training data has missing info or errors, the detection accuracy will drop.
- **Processing Lag:** While we have optimized the speed, analyzing traffic in **real-time** on extremely busy networks is still a challenge and requires very powerful hardware.
- **Black-Box Nature:** Advanced models like XGBoost are great at giving answers, but they don't always explain *why* they flagged a specific activity, making it harder for humans to double-check the logic.

5. Future Work

- **Real-Time Protection:** Moving from analyzing "saved data" to watching **live network traffic** as it happens, so threats can be blocked instantly.
- **Deep Learning:** Testing more advanced models, like **Neural Networks**, to see if they can catch even more complex and hidden hacking patterns.
- **Explainable AI:** Adding tools that explain **why** a specific activity was flagged. This helps security experts understand the system's "thinking" more clearly.
- **Automated Response:** Connecting the system to a firewall so it can **automatically block** an attacker from the moment they are detected, without waiting for a human.
- **Mobile & IoT Security:** Expanding the system to protect smart devices (like smart cameras or watches) and mobile phones, which are often easier to hack.
- **Self-Learning Systems:** Developing a system that **learns on its own** from new attacks without needing a human to retrain it manually every time.

6. Conclusion

In conclusion, this project successfully demonstrates that an AI-driven approach using ensemble learning is a highly effective and scalable solution for modern network security. By transitioning from traditional, static rule-based defenses to a behavioral analysis framework utilizing the CICIDS2017 dataset, the system provides a more proactive shield against sophisticated cyber threats. The implementation of Random Forest and XGBoost proved that combining multiple learning models can significantly enhance detection accuracy while simultaneously reducing the false-positive rates that often lead to alert fatigue in security operations.

Furthermore, the integration of feature optimization techniques confirmed that a focused subset of network attributes, such as Flow Duration and Destination Port, is sufficient to maintain high security standards while reducing computational latency by approximately 40%. This balance of performance and speed ensures that the system is not only accurate but also capable of operating in high-velocity network environments. Ultimately, the project provides a data-driven foundation for more informed security planning and threat hunting, offering a resilient architecture that can adapt to the evolving landscape of global cybersecurity.

REFERENCES

- I referenced this **Kaggle** project for its implementation of machine learning models and data handling techniques at: https://www.kaggle.com/code/chaelsoo/network-intrusion-detection-system
- **CICIDS2017 Dataset:** Sourced from **Kaggle** and the **Canadian Institute for Cybersecurity** for realistic network traffic analysis and attack simulations.
- **Model Frameworks:** Algorithmic logic and ensemble learning implementation strategies adapted from **GeeksforGeeks**.